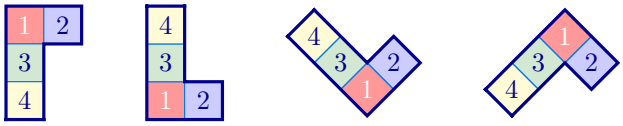
This version of the package is already quite powerful, but it is still in the proof-of-concept phase. There are still bugs and some commands are not yet documented. Several planned features have not been implemented. Everything is subject to change. Comments and suggestions are welcome.

aTableau

Andrew Mathas

Version 2.0.alpha

Contents



1. Introduction

The package provides commands for drawing common objects in algebraic combinatorics and representation theory, together with styling options and a key-value interface for additional customization. Under the hood, everything is drawn using TikZ. These commands can be used either as standalone functions or as part of a `tikzpicture` environment. To use the package, add the following line to your document preamble:

```
\usepackage[options]{atableau}
```

Here, the *options* is an optional comma-separated list of aTableau package options — we describe the possible options below. For example, to make the tableaux and Young diagrams commands use the French convention, by default, you could use:

```
\usepackage[french]{atableau}
```

You can change the default options at any point in your document using the `\aTabset` command. Options can also be used with individual commands.

The aTableau package provides the following commands:

Command	Diagram
\Abacus	An abacus for a partition
\Diagram	A Young diagram for a partition
\Multidiagram	An ℓ-tuple of Young diagrams
\Multitableau	An ℓ-tuple of tableaux
\RibbonTableau	A ribbon tableau
\ShiftedDiagram	A shifted Young diagram
\ShiftedTableau	A shifted tableau
\SkewDiagram	A skew Young diagram
\SkewTableau	A skew tableau
\Tableau	A tableau
\Tabloid	A tabloid

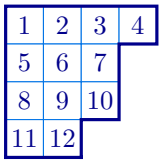
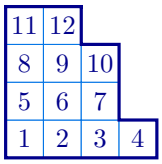
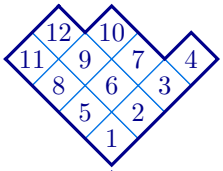
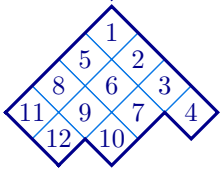
The commands are designed to be easy to use, easy read, and are highly customisable, allowing for the different components of these diagrams. Each command accepts key-value options for modifying the diagrams and optional styling can be applied to their individual components. Each command can be used as a standalone object in a document, or they can be incorporated into a larger `tikzpicture` environment. This makes it possible to use these commands as part of more complicated diagrams.

The general syntax of an aTableau command is:

```
\Command (x,y) [options] {...specifications...}
```

The (x, y)-Cartesian coordinates are optional, being necessary if and only if the diagram is part of a `tikzpicture` environment. Similarly, the key-value *options* control the style and appearance of the final diagram.

The commands are intended to be easy to use. For example, partitions can be entered either as a (weakly decreasing list of non-negative) integers, or using exponential notation, or as a sequence of beta numbers (for the `\Abacus` command), and tableaux are entered as comma separated words. The commands support the common conventions. For example, the following table shows how to draw a Young diagram of shape $\lambda = (4, 3^2, 2)$ using the four tableaux conventions supported by aTableau:

Convention	aTableau command	Diagram
English	`\Tableau[english]{1234,567,89{10},{11}{12}}` (the default)	
French	`\Tableau[french]{1234,567,89{10},{11}{12}}`	
Ukrainian[1]	`\Tableau[ukrainian]{1234,567,89{10},{11}{12}}`	
Australian[2]	`\Tableau[australian]{1234,567,89{10},{11}{12}}`	

[1] Some authors call this the Russian convention.

[2] I have not seen a name for this convention in the literature. Calling this the *Australian convention* seems apt.

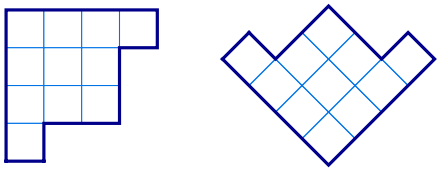
This table shows how the commands provided by aTableau work: there are basic commands for drawing diagrams, and these diagrams can be embellished using optional arguments, which use a key-value interface.

The different conventions for diagrams, and tableaux, are listed in order of (my perception of) their popularity in the literature. By default, the *English* convention is used, so we can omit *english*:

1	2	3	4
5	6	7	
8	9	10	
11	12		

```
1.1
\Tableau{1234,567,89{10},{11}{12}}
```

for the first tableau in the table. These four conventions can be used in exactly the same way with the `\Diagram` command:



```
1.2
\Diagram{4,3,3,1}
\Diagram[ukrainian]{4,3^2,1}
```

This example shows that partitions can either be typed out in full, as a comma-separated list, or using exponential notation, which is usually shorter and easier to read.

The possible options, or keys, for the aTableau commands are described in detail below, with examples. This section emphasises only some of these options together with the fact that all of these keys can be set "gobally" (inside the current LaTeX group), by using `\aTabset{...options...}`. As discussed already, the default options for the document can also be set via `\usepackage[options]{atableau}`.

Many of the aTableau options are specific to the particular diagrams being drawn. All of the options are described in detail in later sections. The following options are common to all of the commands.

align :
Sets the vertical alignment of the diagrams. The possible options are `align=top`, `align=center` and `align=bottom`, which aligns the baseline of the diagram with the top, centre or bottom of the surrounding objects, respectively. By default, aTableau are centered.

name :
Set the prefix for the TikZ *named anchors* for the boxes and beads in aTableau diagrams.

scale, xscale, yscale :
Use scale to rescale all aTableau diagrams. Use xscale to rescale in the x-direction and yscale to rescale in the y-direction.

star style :
Adding a $*$-prefix to elements of an aTableau diagram is a shorthand for changing the style of that element. Use star style to change the default $*$-styling.

tikzpicture :
All aTableau commands are drawn inside a `tikzpicture` environment. Use tikzpicture=keys to set the optional argument of this environment:

`\begin{tikzpicture}[ keys ]...\end{tikzpicture}`.

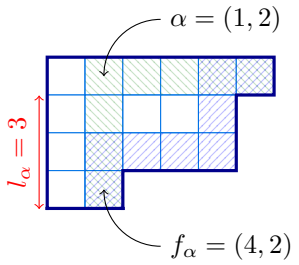
tikz before, tikz after :
Use the tikz before and tikz after keys to inject TikZ code into the `tikzpicture` environment, before and after the diagram drawn by aTableau.

For boolean options, which are set to true or false, the value can typically be omitted. For example, border is the same as border=true (and no border is the same as border=false). In all of the options, American spelling variations, such as *center* (instead of *centre*), *color* (instead of *colour*) et cetera, are tolerated. The names of the default colours used by aTableau are given in Chapter 6.

The next example shows how, how to use the `\Diagram` command inside a `tikzpicture` environment by by supplying Cartesian coordinates.

```
1.3
% \usetikzlibrary{patterns}
\begin{tikzpicture}[
```

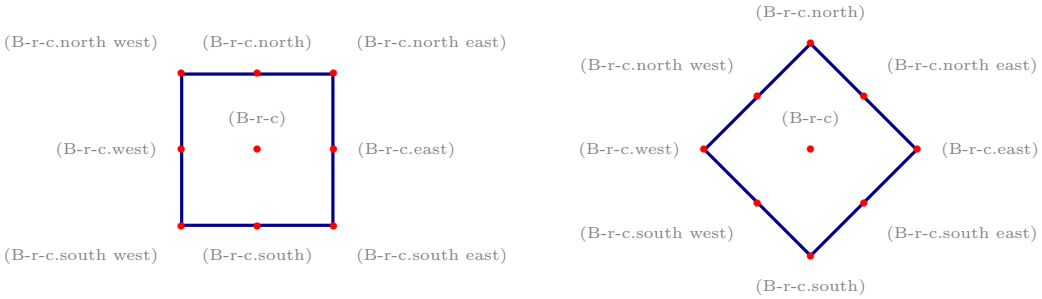
```
      B/.style={pattern=north east lines, pattern color=blue,opacity=0.5},
      G/.style={pattern=north west lines, pattern color=ForestGreen,opacity=0.5},
    ]
    \Diagram(0,0)[ribbons={(G)16ccccrrr, (B)16crrcccr}]{6,5^2,2}
    \draw[->](1.5, 0.5)node[right]{$\alpha=(1,2)$} to [out=180,in=90]  (B-1-2.base);
    \draw[->](1.5,-2.5)node[right]{$f_\alpha=(4,2)$} to [out=180,in=270] (B-4-2.base);
    \draw[red,<->]([xshift=-1mm]B-1-1.south west)
      --node[rotate=90,above]{$l_\alpha=3$}([xshift=-1mm]B-4-1.south west);
  \end{tikzpicture}
```



Most of the commands in the last example are TikZ commands, but most of the picture is drawn by the `\Diagram` command, with the ribbons key specifying two *ribbons*, which are the shaded boxes giving the $(1,2)$-*hook* and $(1,2)$-*rim hook*, respectively. The top of the example defines two TikZ-styles, B and G, which are the styles used by the two ribbons to shade the hooks. The Young diagram is placed at position $(0,0)$ by the `\Diagram` command. The three `\draw` commands are used to add the three arrows to the picture. Please consult the very readable TikZ manual if you would like more details on what the TikZ-commands are doing in this example.

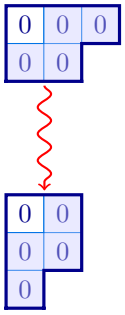
From the viewpoint of aTableau, the most important point of the last example is that the `\Diagram` command can be used inside a `tikzpicture` environment because it is equipped with the Cartesian coordinate $(0,0)$. The example uses the four named coordinates: (B-1-1), (B-1-2), (B-4-1), and (B-4-2). When aTableau draws a tableau, or a diagram, it creates TikZ named coordinates for each of the boxes in the diagram, so that (B-r-c) refers to the box in row r and column c. Giving finer control, the following standard TikZ *anchors* are available:

(1.1)



In the name (B-r-c), r and c should be replaced by the row and column index of a box in the tableau, because these anchors are *only* defined for boxes in the tableau. These *anchors* are a standard part of TikZ. If you change the shape of the node, then the available anchors may change; for more information see the TikZ manual. You can change the B in the name of these anchors to anything you like using the name option. Similar anchors are created for abacus beads.

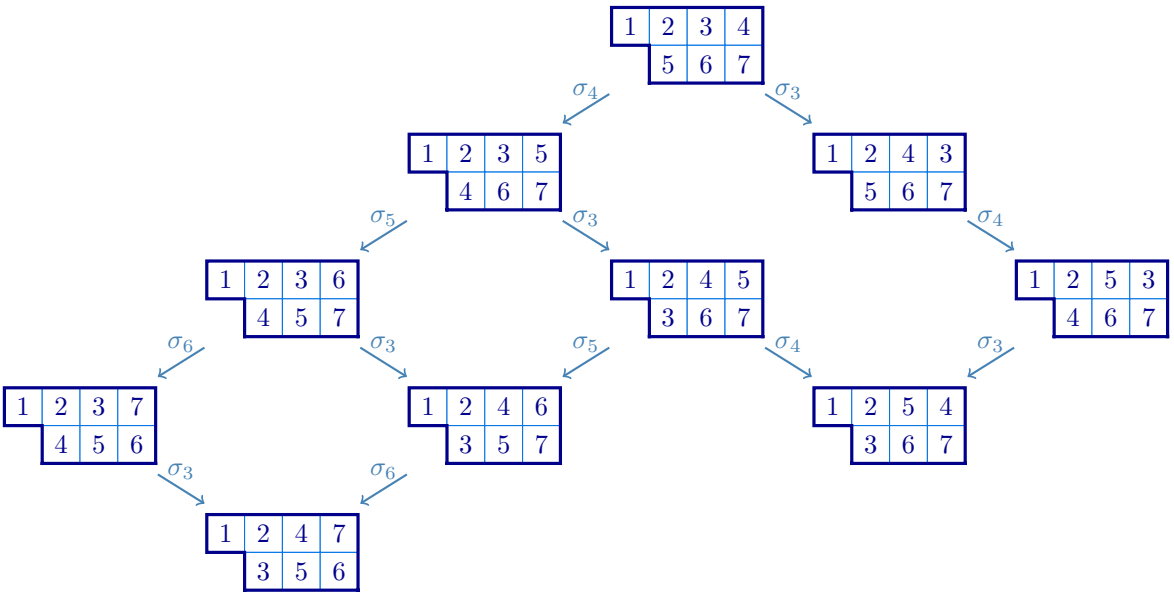
In the example above the Cartesian coordinate $(0,0)$ is necessary because the `\Diagram` command is used inside a `tikzpicture` environment, but the choice of coordinate is not important in the sense that changing the coordinate will not change the picture. Cartesian coordinates become more meaningful when the `tikzpicture` environment has more components. In the next example, notice the use of name=A to make the named coordinates for the first diagram take the form (A-r-c), instead of the default of (B-r-c).



1.4
```
% \usetikzlibrary{decorations.pathmorphing}
\begin{tikzpicture}[wave/.style={thick,red,->,decorate,decoration=snake}]
  \aTabset{ribbon style={fill=blue!20, opacity=0.4}}
  \Diagram(0,0)[e=2,show=residues,ribbons={13crc},name=A]{3,2}
  \Diagram(0,-2.5)[e=2,show=residues,ribbons={12rcr}]{2^2,1}
  \draw[wave]([shift={(0,-0.05)}]A-2-1.south east)
              --([shift={(0,0.05)}]B-1-1.north east);
\end{tikzpicture}
```

The final example in the introduction uses `\ShiftedTableau` inside a *matrix of nodes*:

1.5
```
% \usetikzlibrary{matrix}
\begin{tikzpicture}[scale=0.8, arr/.style={->,SteelBlue, thick}]
  \matrix (M)[matrix of nodes,row sep=4mm,column sep=4mm]{
      &    &    & \ShiftedTableau{1234,567} \\
      &    & \ShiftedTableau{1235,467}
      &    & \ShiftedTableau{1243,567}\\
      & \ShiftedTableau{1236,457}
      &    & \ShiftedTableau{1245,367}
      &    & \ShiftedTableau{1253,467}\\
    \ShiftedTableau{1237,456}
      &    & \ShiftedTableau{1246,357}
      &    & \ShiftedTableau{1254,367}\\
      & \ShiftedTableau{1247,356}\\
   };
   \draw[arr](M-1-4)--node[above]{$\sigma_4$}(M-2-3);
   \draw[arr](M-1-4)--node[above]{$\sigma_3$}(M-2-5);
   \draw[arr](M-2-3)--node[above]{$\sigma_5$}(M-3-2);
   \draw[arr](M-2-3)--node[above]{$\sigma_3$}(M-3-4);
   \draw[arr](M-2-5)--node[above]{$\sigma_4$}(M-3-6);
   \draw[arr](M-3-2)--node[above]{$\sigma_6$}(M-4-1);
   \draw[arr](M-3-2)--node[above]{$\sigma_3$}(M-4-3);
   \draw[arr](M-3-4)--node[above]{$\sigma_5$}(M-4-3);
   \draw[arr](M-3-4)--node[above]{$\sigma_4$}(M-4-5);
   \draw[arr](M-3-6)--node[above]{$\sigma_3$}(M-4-5);
   \draw[arr](M-4-1)--node[above]{$\sigma_3$}(M-5-2);
   \draw[arr](M-4-3)--node[above]{$\sigma_6$}(M-5-2);
\end{tikzpicture}
```



Note that even though the \SkewTableau commands are being used inside a tikzpicture environment they do not need coordinates because the skew tableaux are not explicit components in the environment, being all (implicitly) inside TikZ-nodes.

The following sections describe the commands defined by the aTableau package and how to use them. We start with the \Tableau command because all of the diagram and tableau commands, except for \RibbonTableau , are derived from \Tableau .

2. Tableaux and Young diagrams

Partitions, which are weakly decreasing sequences of non-negative integers, are fundamental objects in algebraic combinatorics and representation theory. Rather than working with sequences of integers, it is often useful to visualise partitions as Young *diagrams*, which are arrays of *boxes*, or *nodes*, in the plane[3]. A *tableau* is a (Young) diagram where the boxes are labelled using some alphabet. Equivalently, a diagram is a tableau with empty labels.

For example, $\lambda = (4, 3, 3, 2, 0, 0, \dots)$ is a partition of 12, since the entries sum to 12. It is customary to omit the zeros when writing partitions and to use exponents for repeated parts, so we can write this partition in *exponential notation* as $\lambda = (4, 3^2, 2)$. Many authors identity a partition $\lambda = (\lambda_1, \lambda_2, \dots)$ with its *diagram*, which is the set $[\lambda] = \{\, (r, c) \,|\, 1 \le c \le \lambda_r \text{ for } r \ge 1 \,\}$. When using the *English convention*, the diagram of λ is visualised as a left justified array of boxes in the plane, where the first row has 4 boxes, the next two lower rows have 3 boxes and the last row has 2 boxes. The *shape* of a diagram is the partition that gives the number of boxes in each row. Diagrams and tableaux drawn using the *French* convention are given by reflecting in a horizontal line above the diagrams, whereas the *Ukrainian* and *Australian* conventions are given by appropriately rotating these diagrams. All of these conventions are used in the literature, with some papers using more than one convention.

2A. **Tableaux.**

This section describes the \Tableau command, which draws tableaux or labelled diagrams. The syntax of this command is:

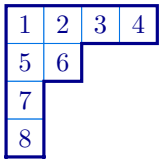
```
\Tableau (x,y) [options] {tableau entries}
```

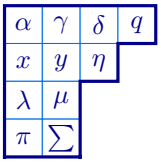
(x, y) :
The Cartesian coordinates (x, y) are needed if and only if the tableau is a component of a tikzpicture environment.

options :
Optional arguments, described below.

tableau entries :
The tableau entries are given as a comma-separated list, with commas separating rows and each *token*, or braced-group of tokens, being in a separate column. As we explain below, each tableau entry can be prefixed by (optional) TikZ-styling specifications.

We show by example how to use the \Tableau command, starting with basic usage and finishing with style. Inside \Tableau , the rows are separated by commas, with the columns given by the "letters" in the word for each row:



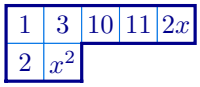


2A.1
```
\Tableau{ 1234, 56, 7, 8 }
\Tableau{ \alpha\gamma\delta q,
          xy\eta,
          \lambda\mu,
          \pi\sum }
```

The tableau specifications can be put on a single line, without any spaces. Alternatively, as shown here, spaces and line breaks can be added to improve readability. The one restriction is that you cannot have blank lines inside a \Tableau command. As this example suggests, by default, the entries in a tableau are typeset in mathematics-mode, but this can be changed using the text nodes option, which is described below.

[3] In this manual, we normally refer to the entries of tableaux and diagrams as *boxes*, rather than *nodes*, to avoid any ambiguity with TikZ-nodes.

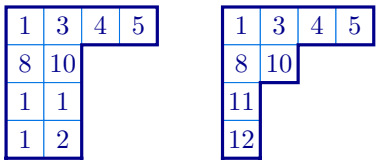
As the examples above show, the tableau entries can be individual characters, or they can be TEX commands. Tableaux frequently contain entries that are not single characters, or given by commands. Such entries can be added to a tableau by enclosing them in braces:



2A.2
```
\Tableau{ 13{10}{11}{2x}, 2{x^2} }
```

There is one additional caveat for braced-entries: any *singleton* braced entry `{...}` in a row needs to be *double-braced*. Double-bracing is only necessary when you have a row that has only one column, and you have a multi-letter column entry. The need for double-bracing is a consequence of how LATEX3 detects braced commas in comma-separated lists.

The following example shows the difference between single and double-bracing for tableau entries.

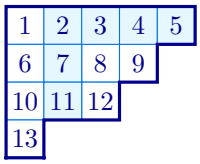


2A.3
```
\Tableau{ 1345, 8{10}, {11}, {12} }
\Tableau{ 1345, 8{10}, {{11}}, {{12}} }
```

The 10's in these tableaux do not need to be doubled-braced because the second row of these tableau has two columns. In contrast, because they are not doubled-braced, the 11 and 12 in the first tableau are both treated as being two characters in different columns in the first tableau. In the second tableau, the 11 and 12 both occupy a single column because they are double-braced.

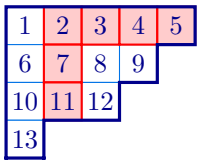
2A.1. *Adding style.* The aTableau package provides several different ways to add TikZ style specifications to the entries in tableaux, and to boxes in diagrams. This is made possible by adding style prefixes to the tableau entries. To take advantage of these styles you will need at least rudimentary knowledge of how TikZ-styling works. The examples in this manual give a good indication of what is possible. For anything more exotic, please consult the TikZ manual.

The simplest way to change the style of an entry in a tableau is to add a ∗-prefix to its tableau specification.



2A.4
```
\Tableau{ 1*2*3*4*5, 6*789, {10}*{11}{12}, {{13}} }
```

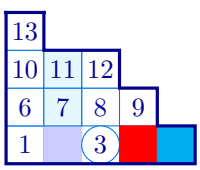
By default, the ∗-entries are given the TikZ-styling `fill=` . You can change the default ∗-style using the star style key:



2A.5
```
\Tableau[star style={fill=red!20,draw=red,thick}]
        { 1*2*3*4*5, 6*789, {10}*{11}{12}, {{13}} }
```

Note that the ∗-styling overrides the default styling of the surrounding boxes. This is because the boxes with the default styling are placed first, in the order that they are entered, followed by boxes with custom styles are placed.

The ∗-syntax is a quick shorthand for adding emphasise some of the boxes in a tableau using a *single* style. It is possible to give every box in the tableau its own style by putting TikZ-styling specifications inside square brackets, `[...]` , before the corresponding letter in the tableau specification. You can mix the ∗-syntax and the `[...]` -style syntax for different entries, although only one of these style settings can be applied to any given box.



2A.6
```
\Tableau[french]{
    1 [blue!20]2 [circle]3 [red]4 [cyan]5,
    6 *7 8 9,
    {10} *{11} {12},
    {{13}} }
```

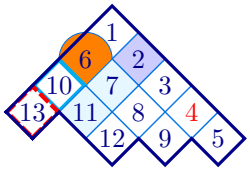
It is not necessary to add spaces between the entries in the way that this example does, which is done only to make it clearer how the style specifications are applied to the different boxes in the tableau.

Omitting some technicalities, each box in a tableau is constructed as a TikZ node, so any TikZ-styling specifications for a node can be used. This example shows that some care is needed when changing the style because simply giving a colour changes both the colour used to fill the box and the text colour, which is why some of tableau entries in the last example appear to be blank. If you are already familiar with TikZ then you probably already know what to do. If not, then here is a short list of useful style specifications for TikZ nodes:

Style	Meaning
`draw` = ⟨colour⟩	Sets the boundary colour of the node
`fill` = ⟨colour⟩	Sets the fill, or background, colour of the node
`font` = ⟨font commands⟩	Sets the font
`text` = ⟨colour⟩	Sets the text colour in the node

In the style settings, any valid LaTeX colour name can be used; see, for example, the `xcolor` manual. In addition, the style specifications `thin`, `thick`, `very thick`, `ultra thick`, `dashed`, `dotted`, ... change the border thickness, and its properties. Subsection 2A.3 below describes how to use these options to customise the tableau style.

If you want to apply "complicated" style settings, such as `draw=cyan,ultra thick` which consists of a comma-separated list of TikZ–style settings, then the entire style must be put inside matching square and curly braces `[{...}]`. This is necessary to ensure that LaTeX does not get confused by the commas in the style setting because commas are used to separate the rows of the tableau.

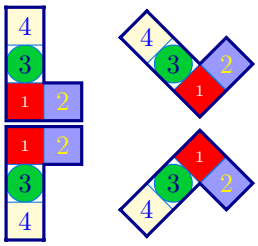


2A.7
```
\Tableau[australian]{
    1 [fill=blue!20]2 3 [text=red]4 5,
    [{circle,fill=orange}]6 *7 8 9,
    [{draw=cyan,ultra thick}]{10} *{11} {12},
    [{dashed,draw=red,ultra thick}]{13} }
```

The last example shows that it is not necessary to double-brace singleton entries when they have a style specification.

This example shows that modifying the borders of a box does adds only minimal emphasis. Use use this sparingly, especially since boxes placed later will redraw the border (see Subsection 2A.5). In addition, the orange circle in this example looks too big, but it is what we asked for because its diameter matches the horizontal width of the box. The next example shows that we get a better result for *Australian* and *Ukrainian* tableaux if we add a *minimum size* specification.

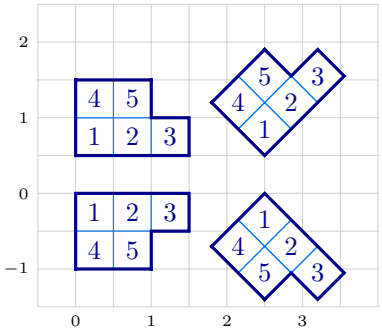
For complex or frequently used styles, we recommend using the `\tikzset` command to define the style outside of the `\Tableau` command. This makes the style both easier to read and easier to change.



2A.8
```
\tikzset{
   B/.style={fill=blue!40, text=yellow},
   G/.style={fill=green!80!blue,circle,minimum size=5mm},
   R/.style={fill=red,text=white, font=\tiny},
   Y/.style={fill=yellow!20, text=blue}
}
\Tableau[french]{[R]1[B]2,[G]3,[Y]4}
\Tableau[ukrainian]{[R]1[B]2,[G]3,[Y]4}
\Tableau[english]{[R]1[B]2,[G]3,[Y]4}
\Tableau[australian]{[R]1[B]2,[G]3,[Y]4}
```

In this example, the TikZ-styles B, G R, and Y are applied to the tableaux entries labelled 1, 2, 3, and 4, respectively. Note the use of *minimum size* to make the circle the right size. The colours used here, and throughout this manual, are for demonstration purposes only: I recommend against such garish choices in any self-respecting document!

2A.2. *Tableau coordinates.* The (x, y)-coordinates are required if, and only if, `\Tableau` command is used inside a `tikzpicture` environment, in which case (x, y) gives the coordinates of the "outside corner" of the box in row 1 and column 1 of the tableau. The following example shows how tableaux are placed using (x, y)-coordinates inside a `tikzpicture` environment. The example also shows that, by default, the tableau boxes are half a unit wide.



2A.9
```
\begin{tikzpicture}[add grid]
  \Tableau(0,0)     [english]   {123,45}
  \Tableau(0,0.5)   [french]    {123,45}
  \Tableau(2.5,0.5)[ukrainian] {123,45}
  \Tableau(2.5,0)   [australian]{123,45}
\end{tikzpicture}
```

2A.3. *Tableau options.* The optional $*$-styles and [...]-styles are the main mechanism for styling individual nodes in a tableau. Using a key-value syntax, you can change aspects of the tableaux produced by the `\Tableau` command. The rest of this section describes these keys, their default values, and gives examples of how to use them.

The options below can be applied to the `\Tableau` command by giving them as a comma-separated list inside square brackets:

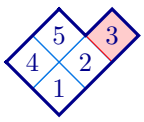
`\Tableau [options] {tableau entries}` or `\Tableau (x,y) [options] {tableau entries}`.

The options can appear in any order, with later options having precedence over earlier ones.

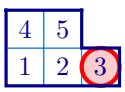
When used inside a `\Tableau` command, the options only affect that particular tableau. Most of these options, or settings, can also be used in the `\aTabset` command, or as optional arguments in `\usepackage[..]{atableau}`, to change the default settings of all tableaux in the same LaTeX group. For example, if you put the command `\aTabset[ukrainian]` in the preamble of your document then, by default, all subsequent tableaux and Young diagrams will be drawn using the Ukrainian convention.

english (default: english)
french
ukrainian
australian

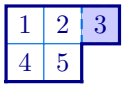
These four (mutually exclusive) options change the convention, or orientation, that is used when drawing the tableaux. Rather than define this precisely, we use the tired and true method in combinatorics of defining by example.



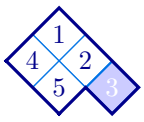
2A.10
```
\tikzset{R/.style={fill=red!20,draw=red}}
\Tableau[ukrainian]{12[R]3,45}
```



2A.11
```
\tikzset{R/.style={fill=red!20, circle, draw=red, ultra
thick}}
\Tableau[french]{12[R]3,45}
```



2A.12
```
\tikzset{B/.style={fill=blue!20, dashed, thick}}
\Tableau[english]{12[B]3,45}
```

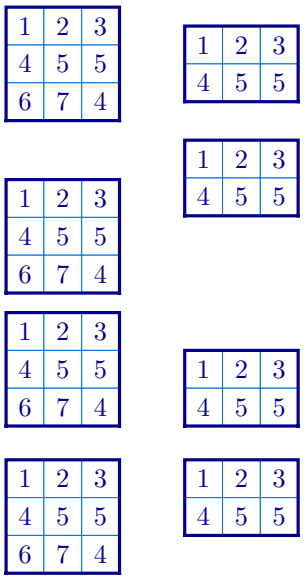


2A.13
```
\tikzset{B/.style={fill=blue!20, thick, text=white}}
\Tableau[australian]{12[B]3,45}
```

By default, the english convention is used to draw tableaux and Young diagrams. The default convention can be changed using `\aTabset` . Capitalised names, `Australian`, `English`, `French` and `Ukrainian`, are also recognised. If, for example, your document is only going to draw french tableaux and diagrams, then you can specify this when you load the package using `\usepackage[french]{atableau}`, or by adding `\aTabset{french}` to your document preamble.

align (default: centre) [accepts: centre/bottom/top]

Use the align key to change the vertical centering of tableaux. By default, tableaux are aligned on their baseline.



2A.14
```
\Tableau{123,455,674}
\Tableau{123,455}
```

2A.15
```
\Tableau[align=top]{123,455,674}
\Tableau{123,455}
```

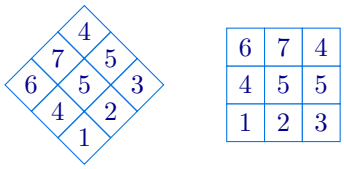
2A.16
```
\aTabset{align=bottom}
\Tableau{123,455,674}
\Tableau{123,455}
```

2A.17
```
\aTabset{align=top}
\Tableau{123,455,674}
\Tableau{123,455}
```

It is not necessary to have the same align keys on each tableau.

border (default: true) [accepts: true/false]
no border (default: false) [accepts: false/true]

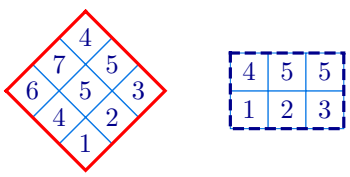
The border and no border options enable and disable the drawing of the border wall around a tableau. As with other boolean keys, border is equivalent to border=true, and no border is equivalent to no border=true. Further, using no border is equivalent to border=false and, similarly, border is equivalent to no border=false.



2A.18
```
\Tableau[ukrainian, border=false]{123,455,674}
\Tableau[french, no border]{123,455,674}
```

border colour (default:) [accepts: any colour]
border style (default: thick,line cap=rect) [accepts: TikZ-styling]

The border colour option sets the colour of the border wall, whereas border style sets the TikZ-styling of the border wall. (So, border style can override border colour.)

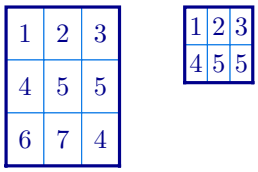


2A.19
```
\Tableau[ukrainian, border colour=red]{123,455,674}

\Tableau[french, border style=dashed]{123,455}
```

The border style *appends* any TikZ-styles to the current styling of the wall.

box height (default: 0.5) [accepts: a decimal giving the height in cm]
box width (default: 0.5) [accepts: a decimal giving the width in cm]

Sets the width and height of the boxes in the tableau. The default height and width of the boxes is 0.5 cm when using the english and french conventions and 0.7012 cm, which is approximately $1/\sqrt{2}$cm, when using the ukrainian and australian conventions. (In all cases, the side length of the squares is 0.5 cm.)

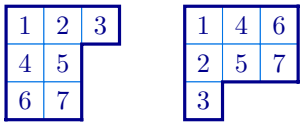


2A.20
```
\aTabset{align=top}
\Tableau[box height=0.7]{123,455,674}
\Tableau[box width=0.3]{123,455}
```

See also the scale, xscale and yscale options.

conjugate (default: false) [accepts: false/true]

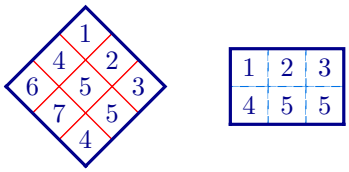
Draws the *conjugate* tableau, where the rows and columns are swapped.



2A.21
```
\Tableau{123,45,67}
\Tableau[conjugate]{123,45,67}
```

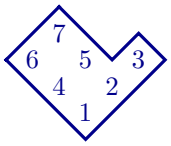
inner colour (default:) [accepts: any colour]
inner style (default: thin,line cap=rect) [accepts: TikZ-styling]

The inner colour option sets the colour of the inner wall, whereas inner style sets the TikZ-styling of the inner wall. (In particular, inner style can be used to override inner colour.)



2A.22
```
\Tableau[australian, inner colour=red]{123,455,674}

\Tableau[inner style=dashed]{123,455}
```

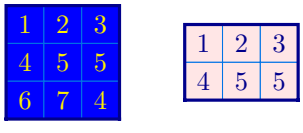
To remove the inner tableau walls use inner colour=none.



2A.23
```
\Tableau[ukrainian, inner colour=none]{123,45,67}
```

box fill (default: none) [accepts: any colour]

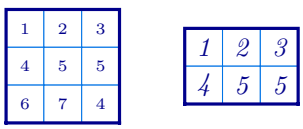
The box fill option sets the background colour of a box in a tableau, just like the fill key for a TikZ-node. If you use a dark fill colour, then you will almost certainly want to change the text colour using the text option – and you may also want to change the colours of the border walls and inner walls using border colour and inner colour, respectively.



2A.24
```
\Tableau[box fill=blue,box text=yellow]{123,455,674}

\Tableau[box fill=red!10]{123,455}
```

box font [accepts:]

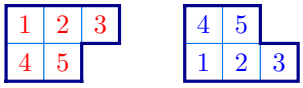
The box font key sets the font used to type the entries of a tableau. By default, tableau entries are typeset as mathematics, so this is mainly useful for changing the font size (because, for example `\bfseries $1$` does not make the 1 bold). It is only when you are using text nodes that font commands like `\itshape` and `\bfseries` will have any effect.



2A.25
```
\Tableau[box font=\tiny]{123,455,674}
\Tableau[text nodes, box font=\itshape]{123,455}
```

box text (default:) [accepts: any colour]

Exactly as for a TikZ-node, use text to set the default text colour for the tableau entries.

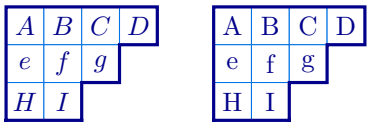


```
\Tableau[box text=red]{123,45}
\Tableau[box text=blue, french]{123,45}
```

2A.26

math nodes (default: true) [accepts: true/false]
text nodes (default: false) [accepts: true/false]

Use the math nodes and text nodes to have the tableau entries typeset as either mathematics of text, respectively. By default, the entries of a tableau are typeset as mathematics.



```
\Tableau[math nodes]{ABCD, efg, HI}      % the default
\Tableau[text nodes]{ABCD, efg, HI}
```

2A.27

name (default: B) [accepts: string]

By default, the boxes can be referenced using the node names (B-1-1), (B-1-2), (B-1-3), . . . , with (B-*r*-*c*) referring to the box in row *r* and column *c*. This feature is most useful when you have several tableaux inside a `tikzpicture` environment because if you only have one tableau, then it is unlikely that you need to change the default prefix for the node names. See (1.1) for a description of the anchor names. Examples are given in Example 1.4 and Example 2A.40.

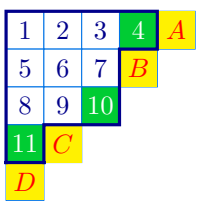
See the introduction to this section for examples and for more information about using node names and anchors.

ribbons [accepts: list of ribbon specifications]
snobs [accepts: list of ribbon specifications]

Use the ribbons key to add a comma-separated list of ribbons to a tableau. Use snobs to add ribbons to the tableau that are placed *after* the border of the diagram is drawn. (Writing *ribbons* backwards gives *snob*bir.)

When adding ribbons, or snobs, you first specify the row and column indices of the *head* of the ribbon, which is the unique node of maximal content in the ribbon, and then give a sequence of r's and c's depending on whether the ribbon moves along a row or column. As always, you have the option of adding style — and you can also specify the contents of each box in the ribbon. For more details about the ribbon specifications, with examples, see Section 2F, which describes the `\RibbonTableau` command.

Use the ribbon style option to change the default style of the ribbons. For example, noting that a box is a ribbon of length 1, the following code uses ribbons of length one (that is, nodes), to highlight the addable nodes of this tableau.

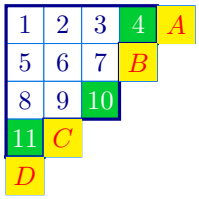


```
\Tableau[star style={fill=green!80!blue,text=white},
        ribbon box={fill=yellow,text=red},
        ribbons={15_A,24_B,42_C,51_D},
]{123*4,567,89*{10},*{11}}
```

2A.28

As this example indicates, ribbons are assumed to be contained inside the diagram of the tableau, so they do not change the border of a tableau.

The default style for a `snobs` ribbon is given by `ribbon style` .

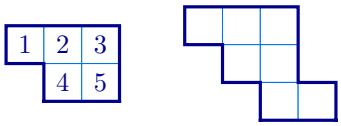



```
\Tableau[star style={fill=green!80!blue,text=white},
         ribbon box={fill=yellow, text=red},
         snobs={15_A,24_B,42_C,51_D},
]{123*4,567,89*{10},*{11}}
```
2A.29

The difference between these two pictures is that in the second picture the nodes in the snobs are drawn over the tableau border, causing the border to disappear for these nodes. For this reason, in most cases you should use ribbons, and avoid snobs.

shifted (default: false) [accepts: true/false]

Set to true for a shifted tableau or shifted diagram. Shifted tableaux, and shifted diagrams, are discussed in more detail in Section 2D, which describes the `\ShiftedDiagram` and `\ShiftedTableau` commands.

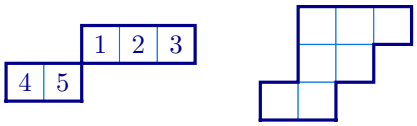



```
\Tableau[shifted]{123,45}
\Diagram[shifted]{3,2^2}
```
2A.30

See Section 2D for the options specific to shifted tableaux and shifted diagrams.

skew (default: (0)) [accepts: a partition]

Specify the skew shape for a skew tableau or skew diagram. Skew tableaux, and skew diagrams, are discussed in more detail in Section 2C, which describes the `\SkewDiagram` and `\SkewTableau` commands.

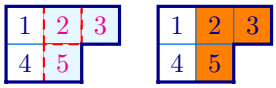



```
\Tableau[skew={2}]{123,45}
\Diagram[skew={2^2,1}]{3,2^2}
```
2A.31

See Section 2C for the options specific to skew tableaux and skew diagrams.

star style (default: fill=) [accepts: TikZ-styling]

Use the star style to add to the style of the starred entries of a tableau. (In TikZ parlance, star style *appends* appends these styles to the default star style.)

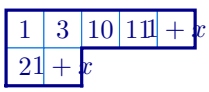



```
\Tableau[star style={text=magenta,
   draw=red, thick, dashed}]{1*2*3,4*5}
\Tableau[star style={fill=orange}]{1*2*3,4*5}
```
2A.32

Note that the tableau border is drawn *after* the star style is applied. You can use snobs to draw on top of the border.

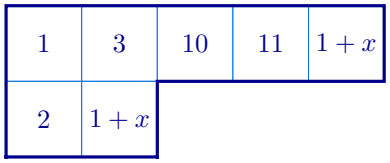
scale (default: 1) [accepts: a decimal number]
xscale (default: 1) [accepts: a decimal number]
yscale (default: 1) [accepts: a decimal number]

By design, boxes in tableaux do not automatically resize to fit their contents. For example, consider:

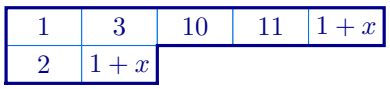



```
\Tableau{ 13{10}{11}{1+x}, 2{1+x} }
```
2A.33

This is unlikely to be the desired output! The options scale, xscale and yscale can be used to rescale the boxes in tableaux and diagrams. The names are slightly misleading because xscale rescales in the direction of increasing column index and yscale rescales in the direction of increasing row index.

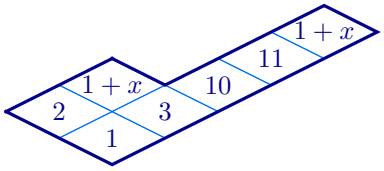



```
\Tableau[scale=2]{ 13{10}{11}{1+x}, 2{1+x} }
```
2A.34

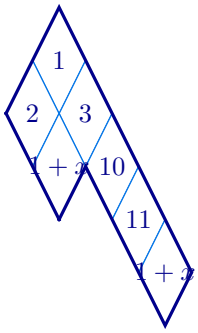



```
\Tableau[xscale=2]{ 13{10}{11}{1+x}, 2{1+x} }
```
2A.35

When using the australian and ukrainian conventions, xscale and yscale scale in both the x direction and the y-direction, reflecting how these tableaux grow with increasing column and row index, respectively.




```
\Tableau[ukrainian, xscale=2]{ 13{10}{11}{1+x}, 2{1+x} }
```
2A.36

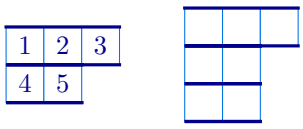



```
\Tableau[australian, yscale=2]{ 13{10}{11}{1+x}, 2{1+x}
}
```
2A.37

The options, scale, xscale, and yscale, affect all aTableau diagrams, including tableaux, Young diagrams and abacuses. If you want to use `\aTabset` to change the default sizes of the diagrams and tableaux in your document, then it is probably better to use the box height and box width options.

tabloid (default: false) [accepts: true/false]

Use the tabloid for tabloids. This option is discussed in more detail in Section 2E, which describes the `\Tabloid` command.

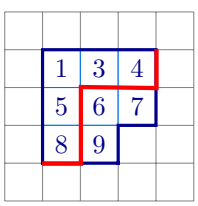



```
\Tableau[tabloid]{123,45}
\Diagram[tabloid]{3,2^2}
```
2A.38

See Section 2E for the options specific to tabloids.

tikz before [accepts: TikZ commands]
tikz after [accepts: TikZ commands]

These keys inject TikZ code into the underlying `tikzpicture` environment, where the tableau is constructed, before and after the `\Tableau` command, respectively.




```
\Tableau[
  tikz before={\draw[help lines,step=0.5](-0.5,-2)grid(2,0.5);},
  tikz after={\draw[ultra thick,red]
                (B-1-3.north east)--(B-1-3.south east)
                --(B-2-1.north east)--(B-3-1.south east)
                --(B-3-1.south west);
  }]{134,567,89}
```
2A.39

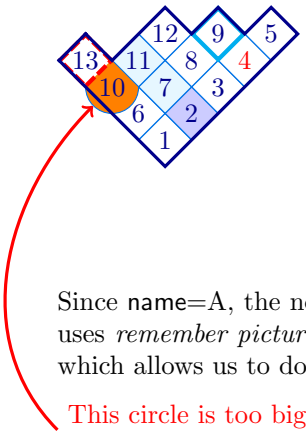
Both before and after hooks are provided because the order in which you place TikZ commands can affect the drawing, primarily because later commands are drawn over the top of earlier ones. Note that the *before*

code cannot use the named nodes B-r-c because this code is executed before these nodes are constructed, which happens when the tableau is drawn. For more examples, see Subsection 2A.5.

tikzpicture

[accepts: TikZ-keys]

This command adds an optional argument to the underlying `tikzpicture` environment, which contains the tableau. The tikzpicture option is ignored when the `\Tableau` command is equipped with (x, y)-coordinates.



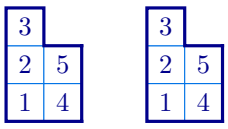
2A.40
```
\Tableau[ukrainian, name=A,
  tikzpicture={remember picture}]
  { 1[fill=blue!20]23[text=red]45,
    6*78[{draw=cyan,ultra thick}]9,
    [{circle,fill=orange}]{10}*{11}{12},
    [{dashed,draw=red,ultra thick}]{13}
  }
```

Since name=A, the nodes in this tableau are referenced as (A-1-1), (A-1-2), (A-1-3), ... As the example uses *remember picture*, other `tikzpicture` environments can reference the node names in last tableau, which allows us to do things like this:

This circle is too big!
Use minimum size=5mm

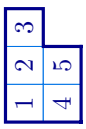
2A.41
```
\tikz[remember picture, overlay]
  \draw[very thick, ->,red]
    (0,0) node[right,align=left]
       {This circle is too big!\\Use minimum size=5mm}
    to [out=135, in=225](A-3-1);
```

As a second example, we can use `tikzpicture={rotate=30}` to try to rotate a tableau by 90 degrees with:



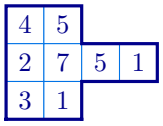
2A.42
```
\Tableau[tikzpicture={rotate=90}]{123,45}
\begin{tikzpicture}[rotate=90]
  \Tableau(0,0){123,45}
\end{tikzpicture}
```

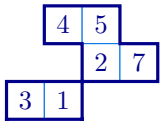
The two tableaux constructions in this example are equivalent. An oddity with both of these examples is that the numbers in the tableaux have not been rotated, which is because the TikZ *rotate* key does not affect component objects, such as nodes, inside a TikZ picture. You can rotate everything in the picture using the *transform canvas* key:



2A.43
```
\Tableau[tikzpicture={transform canvas={rotate=90}}]{123,45}
```

2A.4. *Compositions.* As we have defined them, tableaux are always of *partition* shape, in the sense that the lengths of the rows is a weakly decreasing sequence. In fact, the `\Tableau` command also accepts tableaux of *composition* shape, where the lengths of the rows are not necessarily in decreasing order





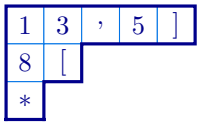
2A.44
```
\Tableau{ 45,2751,31 }
\SkewTableau{1,2}{ 45,27,31 }
```

Tableaux and diagrams should not contain empty rows, and the diagrams should be connected. (For disconnected tableaux and diagrams, see Section 2G.)

2A.5. *Drawing order.* When TikZ constructs a picture, later objects are drawn over the top of earlier ones, sometimes obscuring earlier features. Consequently, for more complicated diagrams and tableaux it helps to know the order in which the different parts of these diagrams are drawn, which is as follows: :

- First, the tableau entries that use the default styling are placed, in the order that appear in the *tableau specifications*.
- Secondly, the styled tableau entries are placed, in the order that they appear in the *tableau specifications*.
- Next, the nodes in any ribbons are placed, again in the order that the ribbons are listed. For each ribbon, first the border of the ribbon is drawn, together with any styling for the ribbon, and then the boxes in the ribbon, together with any styling, are added.
- When enabled by skew border, the skew border is drawn, after which the border of the tableau is drawn.
- Finally, any ribbons specified using snobs are drawn, in the order that they are listed.

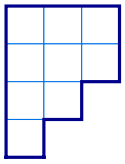
2A.6. *Special characters.* The characters `[]` , `,` , and `*` have special meanings in the *tableau specifications*. If you want these characters to be entries in a tableau, then you need to enclose them in braces:




```
\Tableau{ 13{,}5], 8{[}, {{*}} }
```
2A.45

There is no need to enclose `]` in braces because it only becomes special when it is preceded by a `[` character.

2B. **Young diagrams.** A *Young diagram*, or simply a *diagram*, is an unlabelled tableau. Diagrams can be drawn using the `\Tableau` command:

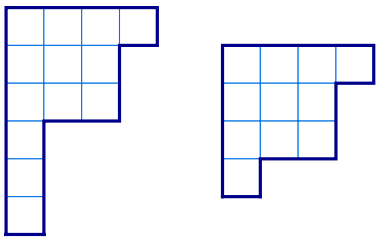


```
\Tableau{~~~,~~~,~~,~}
```
2B.1

This approach works but it is slightly cumbersome and hard to proofread, so aTableau provides the `\Diagram` command. The `\Diagram` command uses almost the same syntax as the `\Tableau` command:

```
\Diagram (x,y) [options] {partition}
```

As with the `\Tableau` command, the (x, y)-coordinates are needed if and only if the diagram is inside a `tikzpicture` environment. The `\Diagram` command allows partitions to be specified either as a comma-separated list of weakly decreasing nonnegative integers, or in *exponential notation*:



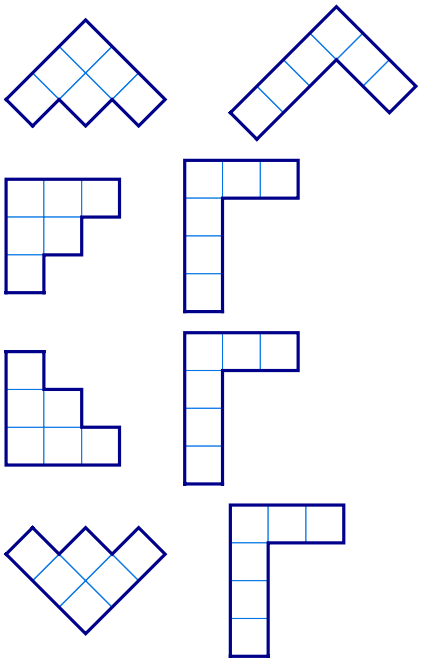
```
\aTabset{align=centre}
\Diagram{4,3^2,1^3}
\Diagram{4,3,3,1}
```
2B.2

Internally, `\Diagram` actually does something like the first example in this section, so all of the options for the `\Tableau` command can be used with `\Diagram` . We do not repeat all of these options in this section and, instead, highlight some of the more interesting ones together with new options that apply specifically to diagrams.

Just like the `\Tableau` command, `\Diagram` supports the four different conventions for diagrams, with english being the default:

english
french

ukrainian
australian (default: english)



```
\Diagram[australian]{3,2,1}
\Diagram[australian]{3,1^3}
```
2B.3

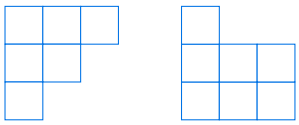
```
% English is the default convention
\Diagram[english]{3,2,1}
\Diagram{3,1^3}
```
2B.4

```
\Diagram[french]{3,2,1}
\Diagram{3,1^3}
```
2B.5

```
\Diagram[ukrainian]{3,2,1}
\Diagram{3,1^3}
```
2B.6

border (default: true) [accepts: true/false]
no border (default: false) [accepts: true/false]

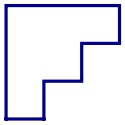
If border is true, then the border of the tableau is drawn. If border is false, then the border of the tableau is not drawn. By default, the border is drawn. The no border option, which is is provided only for completeness, is the inverse of border. That is, no border is equivalent to border=false, and no border=false is equivalent to border=true.



```
\Diagram[border=false]{3,2,1}
\Diagram[no border, french]{3^2,1}
```
2B.7

border only (default: false) [accepts: false/true]

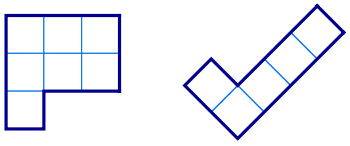
When border only is set to true, only the border of the tableau is drawn, and not the internal walls around the boxes.



```
\Diagram[border only]{3,2,1}
```
2B.8

conjugate (default: false) [accepts: false/true]

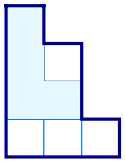
Draws the conjugate diagram, which has the rows and columns interchanged.



```
\Diagram[conjugate]{3,2,2}
\Diagram[ukrainian, conjugate]{2,1^3}
```
2B.9

ribbons [accepts: ribbon specification]
snobs [accepts: ribbon specification]

Use ribbons and snobs to add ribbons to a diagram.



```
\Diagram[french, ribbons={*22*c*r*r}]{3,2^2,1}
```
2B.10

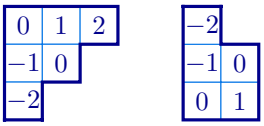
For more details about the ribbon specifications, with examples, see Section 2F, which describes the `\RibbonTableau` command.

show

[accepts: contents,first,hooks,last,residue]

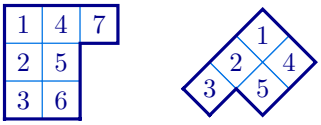
Use show option to draw particular types of tableau of the specified shape. The possible choices are:

- show=contents print the diagram where each box is labelled by its *content*, which is the row index minus the column index.



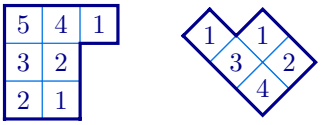
```
\Diagram[show=contents]{3,2,1}
\Diagram[french,show=contents]{2^2,1}
```
2B.11

- show=last print the *last* **standard** tableau of this shape, with respect to the Bruhat order. This is the tableau that has the numbers $1, 2 \ldots, n$ entered in order from top to bottom down successive columns (with appropriate modifications for the non-English conventions).



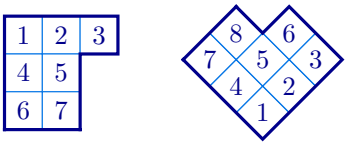
```
\Diagram[show=last]{3,2,2}
\Diagram[australian,show=last]{2^2,1}
```
2B.12

- show=hooks print the diagram where each box is labelled by the corresponding *hook length*. If λ is a partition and λ' is its conjugate partition then the hook length of the box in row i and column j is $\lambda_i - i + \lambda'_j - j + 1$.



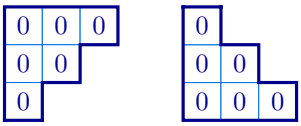
```
\Diagram[show=hooks]{3,2,2}
\Diagram[ukrainian,show=hooks]{2^2,1}
```
2B.13

- show=first print the *first* **standard** tableau of this shape, with respect to the Bruhat order. This is the tableau that has the numbers $1, 2 \ldots, n$ entered in order from top left to right along down successive rows (with appropriate modifications for the non-English conventions).



```
\Diagram[show=first]{3,2,2}
\Diagram[ukrainian, show=first]{3^2,2}
```
2B.14

- show=residues print the diagram where each box is labelled by its *residue*, which is the row index minus the column index modulo some integer e, which must also be supplied.



```
\Diagram[show=residues, e=2]{3,2,1}
\Diagram[french,show=residues, e=3]{3,2,1}
```
2B.15

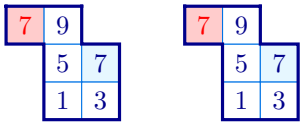
2C. **Skew tableaux and skew diagrams.** A partition μ is *contained* in another partition λ, which is written as $\mu \subseteq \lambda$, if $\mu_k \leq \lambda_k$, for $k \geq 0$. If $\mu \subseteq \lambda$ then the *skew partition* $\lambda/\mu = \{\, (r,c) \in \lambda \,|\, (r,c) \notin \mu \,\}$ is the set of nodes that are in λ and not in μ. In this manual, μ is the *inner shape* and λ/μ is the outer shape. A *skew tableau* is a labelling of the nodes in the diagram of a skew partition. Skew partitions and skew tableau can be drawn using the commands:

```
\SkewDiagram (x,y) [options] {inner shape} {outer shape}
\SkewTableau (x,y) [options] {inner shape} {skew tableau specifications}
```

As with the previous commands, the (x, y)–coordinates should be given if and only if the picture is inside a `tikzpicture` environment. The inner and outer shapes can either be given as a comma-separated list of integers or in exponential notation.

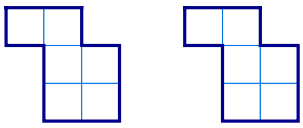
Skew tableau and skew diagrams should always be connected. Use `\Multidiagrams` and `\Multitableau` to draw disconnected diagrams.

In fact, the `\SkewTableau` is just an alias for the `\Tableau`, using the skew key to set the inner shape. For this reason, all of the options for the `\Tableau` command can be used with `\SkewTableau`. The entries in a `\SkewTableau` are specified in exactly the same way as in the `\Tableau` command. In particular, the entries of skew tableaux can, optionally, be given style prefixes, both using a $*$ to add the current star style, or arbitrary TikZ-styling using `[...]`.



2C.1
```
\tikzset{R/.style={fill=red!20,text=red}}
\SkewTableau[french]{1^2}{13,5*7,[R]79}
\Tableau[french,skew={1^2}]{13,5*7,[R]79}
```

Similarly, the `\SkewDiagram` command is an alias for the `\Diagram` command, with a skew key.

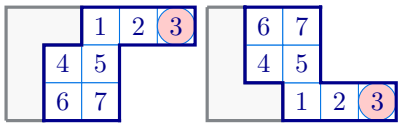


2C.2
```
\SkewDiagram[french]{1^2}{2^3}
\Diagram[french,skew={1^2}]{2^3}
```

Since the `\SkewTableau` and `\SkewTableaua` commands are shortcuts, all of the options for the `\Tableau` and `\Diagram` commands can be used with the `\SkewTableau` and `\SkewDiagram` commands, respectively. In addition, the following options are supported:

skew border (default: false) [accepts: true/false]
no skew border (default: true) [accepts: false/true]

These two options are inverses of each other. When skew border is true, the border of the (inner) skew shape is drawn.

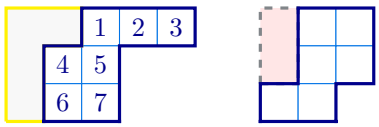


2C.3
```
\tikzset{R/.style={fill=red!20,circle}}
\SkewTableau[skew border]{2,1^2}{12[R]3,45,67}
\SkewTableau[french,skew border]{2,1^2}{12[R]3,45,67}
```

Note that skew border only draws the border of the skew shape and not the boxes inside the inner skew shape. To draw the boxes inside the skew shape use skew boxes

skew border style (default: draw=, fill=, very thick) [accepts: TikZ-styling]

Use skew border style to change the style of the skew border. Since the skew border is only drawn when skew border is set, the skew border style does not have any effect unless skew border has been set to true.



2C.4
```
\SkewTableau[skew border style={draw=yellow},
    skew border]{2,1^2}{123,45,67}
\SkewDiagram[skew border style={dashed,fill=red!10},
    skew border]{1^2}{2^3}
```

To do: Implement the following options

skew boxes (default: false) [accepts: true/false]

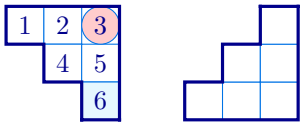
Draw the inner walls in the inside the inner partition of a skew

skew boxes style (default: thin,gray) [accepts: TikZ-styling]

2D. **Shifted tableau and shifted diagrams.** *Strict partitions*, which have strictly increasing parts, and *shifted tableaux*, which are of strict partition shape, appear in several places in representation theory, such as the projective representations of the symmetric groups. These tableaux and diagrams are drawn with row r shifted by r-units along the row, so that the first box in each row has content 0. These diagrams and tableaux can be drawn using the following commands:

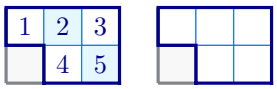
```
\ShiftedDiagram (x,y) [options] {partition }
\ShiftedTableau (x,y) [options] {tableau specification}
```

As usual, the (x, y)-coordinates are necessary if and only if these commands are used inside a `tikzpicture` environment. The partition in a `\ShiftedDiagram` can be given using exponential notation and the tableau specification for a `\ShiftedTableau` can include the usual optional style prefixes.



2D.1
```
\tikzset{R/.style={fill=red!20,circle}}
\ShiftedTableau{12[R]3,45,*6}
\ShiftedDiagram[french]{3,2,1}
```

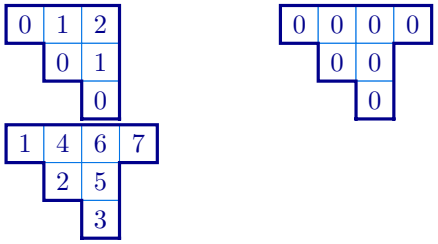
In the literature, shifted tableaux and shifted diagrams almost always have strict partition shape, however, this is not enforced by these commands. Under the hood, the `\ShiftedTableau` is the `\Tableau` command with the shifted option set to true. Similarly, `\ShiftedDiagram` is the same as using the `\Diagram` with the shifted option. Consequently, all of the options for the `\Tableau` and `\Diagram` commands can be used with `\ShiftedTableau` and `\ShiftedDiagram`, respectively. In particular, options like skew wall can be used to highlight the shifted part of the diagram



2D.2
```
\ShiftedTableau[skew border]{1*23,4*5}
\ShiftedDiagram[skew border]{3,2}
```

The show key also works as expected:

To do: Make show work properly for skew, shifted and multi tableaux and diagrams.

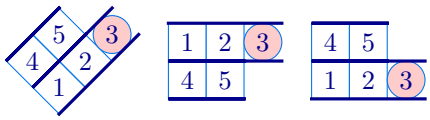


2D.3
```
\ShiftedDiagram[show=contents]{3,2,1}
\ShiftedDiagram[show=residues,e=2]{4,2,1}
\ShiftedDiagram[show=last]{4,2,1}
```

2E. **Tabloids.** A *tabloid* is an equivalence class of tableau, where two tableaux are equivalent if they have the same entries in each row, up to a permutation. In the literature, tabloids are usually drawn with lines above and below each row, and without side borders. Tabloids can be drawn with the `\Tabloid` command.

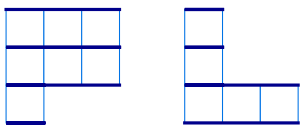
```
\Tabloid (x,y) [options] {partition }
```

The `\Tabloid` command functions in the same way as the `\Tableau` command, except that it draws tabloids. In fact, the `\Tabloid` is a special case of the `\Tableau` command with the tabloid option set.



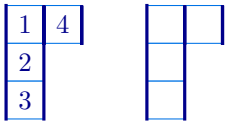
2E.1
```
\tikzset{R/.style={fill=red!20,circle,minimum size=5mm}}
\Tabloid[ukrainian]{12[R]3,45}
\Tabloid[english]  {12[R]3,45}
\Tabloid[french]   {12[R]3,45}
```

The aTableau package does not provide a dedicated command for tabloid diagrams, however, they are can be drawn using the `\Diagram` command together with the tabloid option:



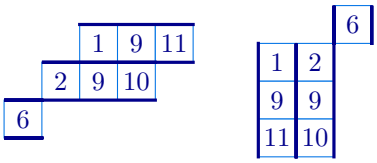
2E.2
```
\Diagram[tabloid]{3^2,1}
\Diagram[tabloid,french]{3,1^2}
```

Some authors work with *column tabloids*. Such tableaux do not have a dedicated aTableau command, however, they can be drawn using the conjugate option:



2E.3
```
\Tabloid[conjugate]{123,4}
\Diagram[conjugate,tabloid]{3,1}
```

Similarly, skew tabloids can be drawn using the skew option.



2E.4
```
\aTabset{align=center}
\Tabloid[skew={2,1}]{19{11},29{10},6}
\Tabloid[conjugate,skew={1^2}]{19{11},29{10},6}
```

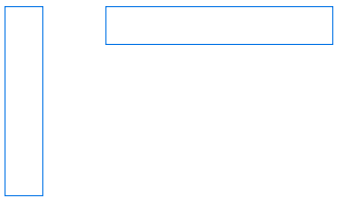
2F. **Ribbon tableaux.** A *ribbon* in a tableau, or a diagram, is a connected strip of boxes R that are totally ordered by their contents. (Recall from Section 2B that the content of the box in row a and column b is $b-a$.) Therefore, a ribbon does not contain a ⊞-square, and if $(a,b) \in R$ then at most one of $(a+1,b)$ and $(a,b-1)$ belongs to R. The *head* of a ribbon R is the unique node of maximal content. A *ribbon tableau* is a tableau that is tiled by ribbons. As a box is a ribbon of length 1, so every tableau is a ribbon tableau.

```
\RibbonTableau (x,y) [options] {ribbon specifications}
```

Like the other commands, the (x,y)-coordinates are optional and are required if and only if the diagram is being used as part of a `tikzpicture` environment. All options for the `\Tableau` command can be used for the ribbon command, so we refer to Section 2A for a description of the available options.

2F.1. *Ribbon specifications.* Unlike the `\Tableau` command, the entries of a ribbon tableau are given by ribbon specifications (rather than tableau) specifications. Ribbon specifications are also used by the ribbons and snobs options discussed in Subsection 2A.3.

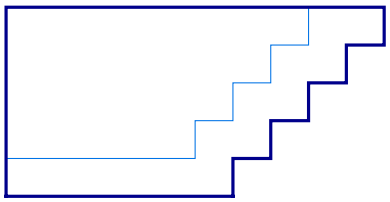
To understand the ribbon specifications, observe that if (a,b) is the head of a ribbon, then we can walk along the ribbon by specifying whether the row index increases or the column index decreases. That is, A ribbon is uniquely determined by specifying its head (a,b) together with a sequence of r's and c's that indicate when the row index increases, or the column index decreases, respectively. For example, vertical and horizontal ribbons are given by a sequences of r's and c's, respectively:



2F.1
```
\aTabset{align=top, no border}
\RibbonTableau{11rrrr}
\RibbonTableau{16ccccc}
```

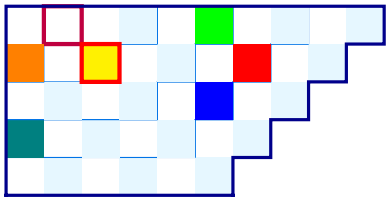
The border of a ribbon tableau is the smallest (skew) partition that contains all of its ribbons. As the last example shows, the no border key disables drawing of the ribbon tableau border because, otherwise, we would not be able to see the ribbon border in these examples since the ribbon is the full diagram in these two cases.

The following diagram shows a see-sawing ribbon with head in row 1 and column 10, and tail in row 5 and column 1.



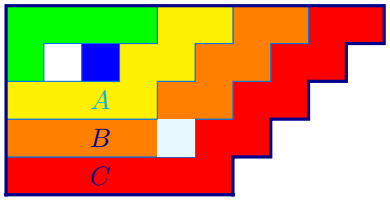
2F.2
```
\RibbonTableau{1{10}crcrcrcrccccc}
```

As with tableau entries, each box in a ribbon can be preceded with a style specification, either using `*` for a star style or using `[...]` for more customised style specifications.



2F.3
```
\RibbonTableau{
    *1{10}c*rc*rc*rc*rc*cc*cc,
    *18c[fill=red]rc[fill=blue]rc*rc*cc[fill=teal]c,
    [fill=green]16c*rc*rc*cc,
    *14c[{draw=purple,ultra thick}]cc[fill=orange]r,
    [{fill=yellow,draw=red,ultra thick}]23
}
```

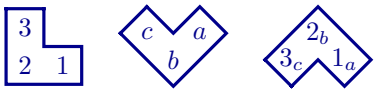
In particular, the yellow box in this example highlights that boxes are ribbons of length 1. Most of the time, you want to style the entire ribbon rather than styling the individual boxes in the ribbon. Optionally, the style for the entire ribbon can be given inside parentheses (...) at the *start* of the ribbon. Unlike the other style specifications, the ∗-shorthand for the star style cannot be applied to an entire ribbon.



2F.4
```
\RibbonTableau{
    (fill=red)1{10}crcrcrcrccc_Ccc,
    (fill=orange)18crcrc*rcc_Bcc,
    (fill=yellow)16crcr[cyan]c_Acc,
    (fill=green)14cccr,
    (fill=blue)23
}
```

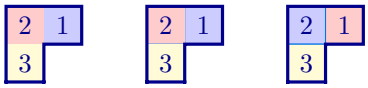
Any styles for a ribbon are applied to the region bounded by the closed path given by the border of the ribbon. The ribbon style is applied first, after which any styled boxes in ribbon are drawn. So, as this example shows, the optional style of any box in the ribbon takes precedence over the style of the full ribbon.

Finally, it is often necessary to label some of the boxes in the ribbon. Optionally, text can be added to the boxes in a ribbon by supplying it as a *subscript* to the corresponding entry in the ribbon specification:



2F.5
```
\RibbonTableau[french]    { 12_1 c_2 r_3 }
\RibbonTableau[ukrainian] { 12_a c_b r_c }
\RibbonTableau[australian]{ 12_{1_a} c_{2_b} r_{3_c} }
```

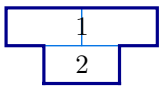
Here we have put spaces between the entries for the different boxes in the ribbon for clarity. There are often many ways to draw the same ribbon tableau, especially if the entries have styling.



2F.6
```
\tikzset{Fill/.style={fill=#1!20!white},
  B/.style={Fill=blue}, R/.style={Fill=red},
Y/.style={Fill=yellow}}
\RibbonTableau{ [B]12_1  [R]c_2   [Y]r_3}
\RibbonTableau{ [B]12_1, [R]11_2, [Y]21_3 }
\Tableau{       [B]2     [R]1,    [Y]3 }
```

As shown, the easiest way to draw this particular tableau is using the `\Tableau` command.

There is no dedicated syntax for putting a label on the boundary between two boxes in the diagram, but this can be done using the named anchors of (1.1).



2F.7
```
\RibbonTableau[shifted, tikz after={
    \node at (B-1-2.east){$1$};
    \node at (B-2-2.east){$2$};
}]{ 14c, 12c, 23c }
```

We have now seen the full ribbon specifications. To summarise:

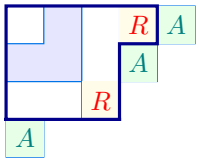
- The ribbon specification starts with optional TikZ-styling for the ribbon, enclosed in (...).
- The first entry in the ribbon is given as ab, if the head of the ribbon is in row a and column b. The remaining entries in the ribbon are specified by a sequence of r's and c's, depending on whether the next box in the ribbon is given by increasing the row index, or decreasing the column index, respectively.

- Optionally, each box in the ribbon, including the head, can be given TikZ-styling by prefixing it with a $*$, which gives the ribbon star style, or by giving TikZ-styling keys inside square brackets [...].

Finally, when drawing the border, ribbons in a `\RibbonTableau` are assumed to be contained in the smallest Young diagram that contains the ribbons. This said, it is up to you to ensure that the boxes in the ribbon not have negative row or column indices. Inside a `\Diagram` or `\Tableau` command, any ribbons added using ribbons, or snobs, do not affect the border of the diagram or tableau, and it is your responsibility to ensure that the boxes in the ribbon are inside the diagram or tableau. This said, it is a useful feature to be able to place boxes outside of the diagrams, which can be done using ribbons or snobs as these do not contribute to the border of a ribbon tableaux.

ribbons [accepts: ribbon specification]
snobs [accepts: ribbon specification]

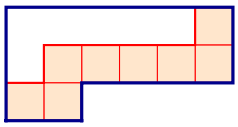
You can add ribbons and snobs to a `\RibbonTableau` in exactly the same way that you add them to `\Tableau`. At first sight, this seems unnecessary because ribbon tableaux are composed of ribbons, however, the point is that any ribbon added to a tableau using ribbons or snobs is not assumed to be inside the border of the tableau.



2F.8
```
\tikzset{A/.style={fill=green!10, text=teal},
  R/.style={fill=yellow!10, text=red}}
\RibbonTableau[ribbons={[A]15_A,[A]24_A,[A]41_A,
[R]14_R,[R]33_R}]
  {11, (fill=blue!10)12rc, 14crrcc}
```

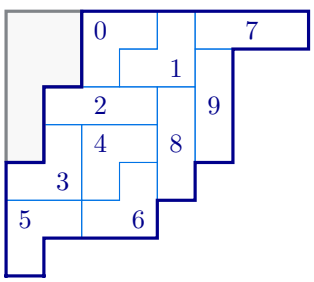
ribbon style (default: draw=, thin) TikZ-styling
ribbon box TikZ-styling

Use the ribbon style and ribbon box options to append to the default styling of the ribbons and the boxes in the ribbons, respectively. By default, ribbons have a inner wall and the boxes in ribbons have no styling.

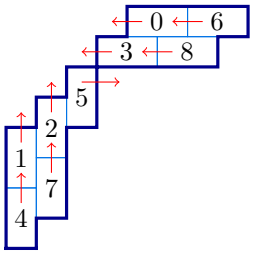


2F.9
```
\RibbonTableau[ribbon style={draw=red, thick},
    ribbon box={fill=orange!20, draw=red}]{16rccccrc}
```

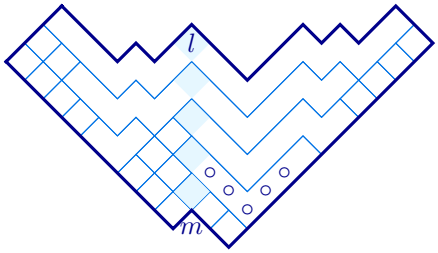
2F.2. *Some ribbed examples.* We close this section showing how to draw some tableaux that appear in the literature.



2F.10
```
\RibbonTableau[skew={2^2,1^2},skew border]{
   14c_0r,15r_1c, 18c_7c,26r_9r,
   34c_2c,35r_8r,42r_3c, 44c_4r,
   54r_6c,62c_5r
}
```



2F.11
```
\RibbonTableau[,skew={4,3,2,1},scale=0.8,
    tikz after = {
      % node labels
      \foreach \r/\c/\l in {1/6/0, 1/8/6, 2/5/3, 2/7/8}
        { \node at (B-\r-\c.west){\l};}
      \foreach \r/\c/\l in {5/1/1, 4/2/2, 3/3/5, 7/1/4, 6/2/7}
        { \node at (B-\r-\c.south){\l};}
      % arrows
      \foreach \a/\b in {1/5, 1/7, 2/4, 2/6}
        { \draw[red,->](B-\a-\b.center)--++(-0.4,0); }
      \draw[red,->](B-3-3.center)--++(0.5,0);
      \foreach \a/\b in {5/1, 7/1, 4/2, 6/2}
        { \draw[red,->](B-\a-\b.center)--++(0,0.4); }
    }
]{
    16c,18c, 25c, 27c, 33r, 42r, 51r, 62r,71r
}
```

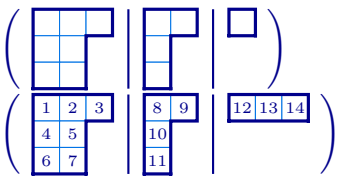


2F.12
```
\RibbonTableau[skew={1^2}, ukrainian, scale=0.7,
  ribbons={(draw=none)21_m}
]{
    12,16c_\circ c_\circ c_\circ r_\circ r_\circ *r,
    18crcccrr*r,19,1{10},1{11},1{12},
    22,29crcccrr*rccrcrr,
    2{12}ccrcrcccrr*r_1ccrcrrr,
    31,*32,41,42,51,52,53,
    62cr,81,91,{10}1,{10}2,{11}1,{11}2
}
```

2G. **Multidiagrams and multitableaux.** Multitableaux and multidiagrams are ℓ-tuples of tableaux and diagrams, respectively, that appear in many places in representation theory, such as when considered representations of wreath products and cyclotomic Hecke algebras. For convenience, aTableau provides the `\Multidiagram` and `\Multitableau` commands for drawing these diagrams, even though it is not difficult to construct such tableaux and diagrams using the `\Tableau` and `\Diagram` commands.

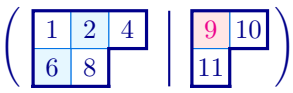
```
\Multidiagram (x,y) [options] {multipartition specifications}
\Multitableau (x,y) [options] {multitableau specifications}
```

Our preferred notation for a multipartition $\boldsymbol{\lambda}$ is to write $\boldsymbol{\lambda} = (\lambda^{(1)}|\lambda^{(2)}|\dots|\lambda^{(\ell)})$, where $\ell \geq 1$ is the *level* and the partitions $\lambda^{(1)},\dots,\lambda^{(\ell)}$ are the *components* of $\boldsymbol{\lambda}$. Similarly, a multitableau is written as $\mathsf{t} = (\mathsf{t}^{(1)}|\dots|\mathsf{t}^{(\ell)})$, where the tableaux $\mathsf{t}^{(1)},\dots,\mathsf{t}^{(\ell)}$ are the *components* of t. Accordingly, in the `\Multidiagram` and `\Multitableau` commands, the pipe symbol `|` is used to separate components:



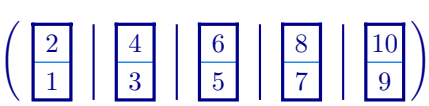
2G.1
```
\aTabset{scale=0.7}
\Multidiagram{3,2^2|2,1,1|1}
\Multitableau[box font=\tiny]
    { 123,45,67|89,{{10}},{{11}}|{12}{13}{14} }
```

As shown, the `\Multidiagram` command accepts exponential notation for partitions. The entries in a `\Multitableau` can be given optional styling specifications, exactly as inside the `\Tableau` command.



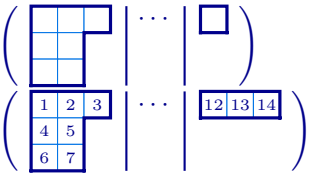
2G.2
```
\tikzset{R/.style={fill=red!10,text=magenta}}
\Multitableau{1*24,*68|[R]9{10},{{11}}}
```

Multitableaux and multidiagrams can, in principle, have arbitrary level. In practice, the level is constrained by the page width.



2G.3
```
\Multitableau[french]{1,2|3,4|5,6|7,8|9,{{10}}}
```

Both the \Multitableau and \Multidiagram commands accept a special component, ... , that is used to add dots between the separators:

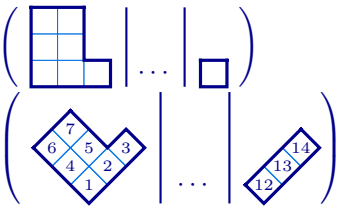


```
\aTabset{scale=0.7}
\Multidiagram{3,2^2|...|1}
\Multitableau[box font=\tiny]
    {123,45,67|...|{12}{13}{14}}
```

The \Multitableau and \Multidiagram commands accept most of the options of the \Tableau and \Diagram commands, together with a few options that are specific to multitableaux and multidiagrams. This section describes the new options, and highlights how some of the other options are used. Currently, ribbons and snobs are *not* supported for multitableaux and multidiagrams.

english
french
ukrainian
australian (default: english)

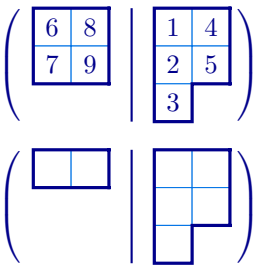
All four tableaux conventions are supported for multitableaux and multidiagrams.



```
\aTabset{scale=0.7}
\Multidiagram[french]{3,2^2|...|1}
\Multitableau[ukrainian, box font=\tiny]
    {123,45,67|...|{12}{13}{14}}
```

conjugate

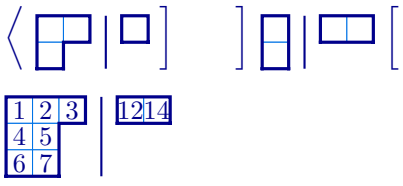
The conjugate option prints the conjugate mutlitableaux and multidiagrams. Conjugation for multitableaux and multidiagrams reverses the order of the components and then conjugates all of the component diagrams.



```
\Multitableau[conjugate]{ 123,45 | 67,89}
\Multidiagram[conjugate]{ 3,2 | 1^2 }
```

delimiters (default: ()) [accepts: pair of LaTeX delimiters]
left delimiter (default: () [accepts: a LaTeX delimiter]
right delimiter (default:)) [accepts: a LaTeX delimiter]

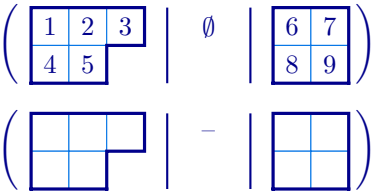
The delimiter options control the delimiters that put on the left and right of multitableaux and multidiagrams. These options accept any LaTeX delimiter. Setting the left delimiter or right delimiter to nothing removes the delimiter.



```
\aTabset{scale=0.7}
\Multidiagram[left delimiter=\langle,
   right delimiter={]}]{2,1|1}
\Multidiagram[delimiters={]}{[}]{1^2|2}
\Multitableau[left delimiter=,
 right delimiter=] {123,45,67|{12}{14}}
```

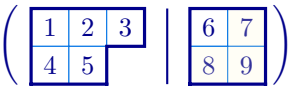
empty (default: \emptyset) [accepts: LaTeX]

The empty option determines what is printed when a component diagram in a multitableau or multidiagram is empty. By default the empty set symbol ∅ is printed.



2G.8
```
\Multitableau{ 123,45 | | 67,89}
\Multidiagram[empty=\textendash]{ 3,2 | | 2^2 }
```

ribbons [accepts: a |-separated list of ribbon specifications]
snobs [accepts: a |-separated list of ribbon specifications]



2G.9
```
\tikzset{Y/.style={fill=yellow!20,opacity=0.2}}
\Multitableau[ribbons={|(Y)12rc}]{ 123,45 | 67,89}
```

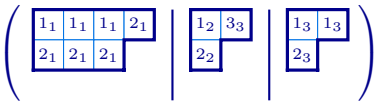
name (default: B) [accepts: character]

The name option is used to change the default prefix for the box node names. Unlike in a tableau or diagram, the box names in multitableaux and muiltidiagrams take the form B-k-r-c, where k is the component index, r is the row index, and c is the column index of the box.

rows [accepts: decimal number]

The rows option sets the number of rows covered by the delimiters in multitableaux and multidiagram. By default, the \Multitableau and \Multidiagram commands choose the size of the delimiters to match the component diagrams, which may not be what you want especially when using the australia and ukrainian conventions. The value of rows can be a decimal number.

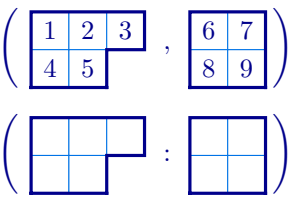
2G.10
```
\Multidiagram[ukrainian,scale=0.5]{3,2^2|...|2,1^2}

\Multidiagram[ukrainian,scale=0.5,
rows=4.3]{3,2^2|...|2,1^2}
```



2G.11
```
\Multitableau[rows=3,scale=0.8,box font=\tiny]
    {{1_1}{1_1}{1_1}{2_1},{2_1}{2_1}{2_1} |
     {1_2}{3_3},{{2_2}} |
     {1_3}{1_3},{{2_3}}}
```

separator (default: —) [accepts: character]

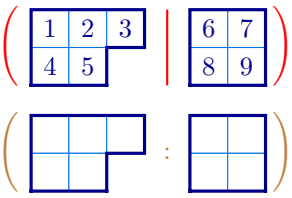
The separator determines what is printed between the component diagrams in multitableaux and multipartition. By default, separator=— draws a straight line between the components, but any character can also be used.



2G.12
```
\Multitableau[separator={,}]{ 123,45 | 67,89}

\Multidiagram[separator=:]{ 3,2 | 2^2 }
```

separator colour (default:) [accepts: LaTeX colour]

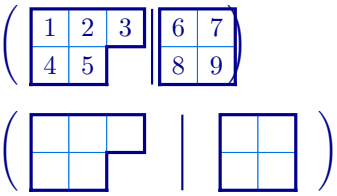
The separator colour sets the colour of the delimiters and the separator between component diagrams. By default, the separators are the same colour as the diagram border.



```
\Multitableau[separator colour=red]{ 123,45 | 67,89}

\Multidiagram[separator color=brown, separator=:]{ 3,2
| 2^2 }
```
2G.13

separation (default: 0.3) [accepts: decimal (distance in cm)]

Use separation to change the distance between the separators and the tableaux.

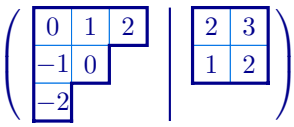


```
\Multitableau[separation=0.1]{ 123,45 | 67,89}

\Multidiagram[separation=0.5]{ 3,2 | 2^2 }
```
2G.14

show [accepts: contents,first,hooks,last,residues]

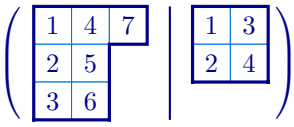
Use show option to draw particular types of tableau of the specified shape. The possible choices are:

- show=contents print the diagram where each box is labelled by its *content*, which is the row index minus the column index.



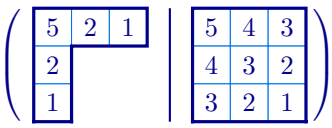
```
\Multidiagram[show=contents,
charge={0,2}]{3,2,1|2^2}
```
2G.15

- show=last print the *last* **standard** tableau of this shape, which has the numbers $1, 2 \ldots, n$ entered in order from top to bottom down successive columns, and left to right along components (with appropriate modifications for the non-English conventions).



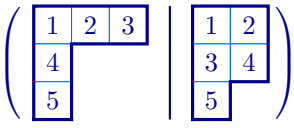
```
\Multidiagram[show=last]{3,2,2|2^2}
```
2G.16

- show=hooks print the diagram where each box is labelled by the corresponding *hook length*. If λ is a partition and λ' is its conjugate partition then the hook length of the box in row i and column j is $\lambda_i - i + \lambda'_j - j + 1$.



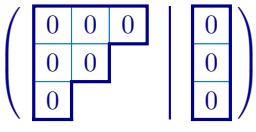
```
\Multidiagram[show=hooks]{3,1^2|3^3}
```
2G.17

- show=first print the *first* **standard** tableau of this shape, which has the numbers $1, 2 \ldots, n$ entered in order from top left to right along down successive rows (with appropriate modifications for the non-English conventions).

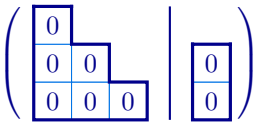


```
\Multidiagram[show=first]{3,1^2|2^2,1}
```
2G.18

- show=residues print the diagram where each box is labelled by its *residue*, which is the row index minus the column index modulo some integer e, which must also be supplied.

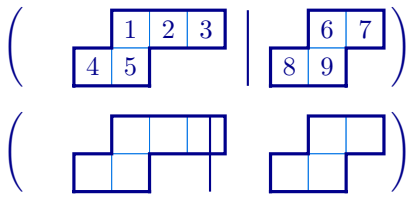


```
\Multidiagram[show=residues, e=2]{3,2,1|1^3}
```
2G.19



```
\Multidiagram[french,show=residues,
e=3,charge={1,2}]{3,2,1|1^2}
```
2G.20

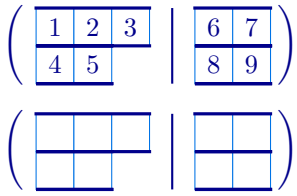
skew a |-separated list of partitions



```
\Multitableau[skew={2,1|1}]{ 123,45 | 67,89}

\Multidiagram[skew={1|2,1}]{ 3,2 | 2^2 }
```

tabloid

Use tabloid for multitabloid diagrams and tableaux.



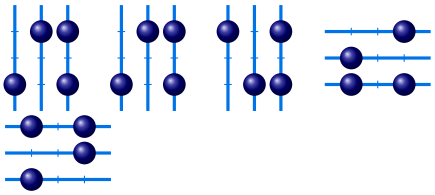
```
\Multitableau[tabloid]{ 123,45 | 67,89}
\Multidiagram[tabloid]{ 3,2 | 2^2 }
```

xoffsets (default: 0) [accepts: command separated list of x-offsets]
yoffsets (default: 0) [accepts: command separated list of x-offsets]

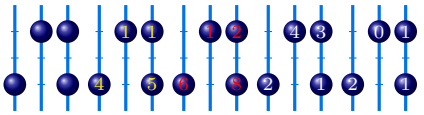
3. Abacuses

An abacus

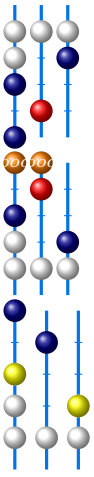
```
\Abacus (x,y) [options] {number of runners} {bead specifications}
```



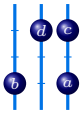
```
\Abacus{3}{5,4,1,1}
\Abacus[south]{3}{5,4,1,1}
\Abacus[north]{3}{5,4,1,1}
\Abacus[east]{3}{5,4,1,1}
\Abacus[west]{3}{5,4,1,1}
```



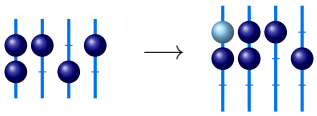
```
\Abacus{3}{5,4,1,1}
\Abacus[show=shape,bead text=yellow]{3}{5,4,1,1}
\Abacus[show=beta,bead text=red]{3}{5,4,1,1}
\Abacus[show=rows]{3}{5,4,1,1}
\Abacus[show=residue]{3}{5,4,1,1}
```



```
\tikzset{R/.style={ball color=red},
         W/.style={ball color=white},
         O/.style={ball color=orange}
}


\Abacus{3}{5,[R]4,1^2,[W]0^4}

\Abacus[north]{3}{[O]5^2_{ooo},[R]4,1^2,[W]0^4}

\Abacus[north, scale=1.2, star style={ball
color=yellow}]{3}{5,4,*1^2,[W]0^4}
```



```
\Abacus[bead text=white, bead
font=\tiny]{3}{5_a,4_b,1_c,1_d}
```



```
\aTabset{align=centre}
\tikzset{B/.style={ball color=LightSkyBlue}}
\Abacus[beta numbers]{4}{6,4,3,1,0}
      $\longrightarrow$
\Abacus[beta numbers]{4}{7,5,4,2,1,[B]0}
```

4. Dynkin quivers?

5. Braid diagrams and KLRW diagrams?

6. Colours

The aTableau package defines and uses the following colours:

Colour	Name	HTML
	aTableauMain	00008B
	aTableauInner	0073E6
	aTableauSkew	818589
	aTableauSkewFill	F8F8F8
	aTableauStarStyle	E6F7FF

All of these colours are defined as HTML colours using the `\definecolor` command provided by `xcolor`.

It is possible to change the colours used in the diagrams created by aTableau by changing these colours. However, we recommend using the key-value interface instead.

7. Feature requests and bug reports

Please make any feature requests and bug reports on the package github page

github.com/AndrewMathas/aTableau/.

Please give as much detail as possible when making such requests.

Bug reports must be a accompanied by a *minimal working example*, which is the smallest possible amount of LaTeX code that compiles and demonstrates the problem. This is necessary because if I cannot replicate your problem, then it is I unlikely that I will be able to fix it.

Index